

- OPD1: si caricano tutti i tavoli (frequenza media)

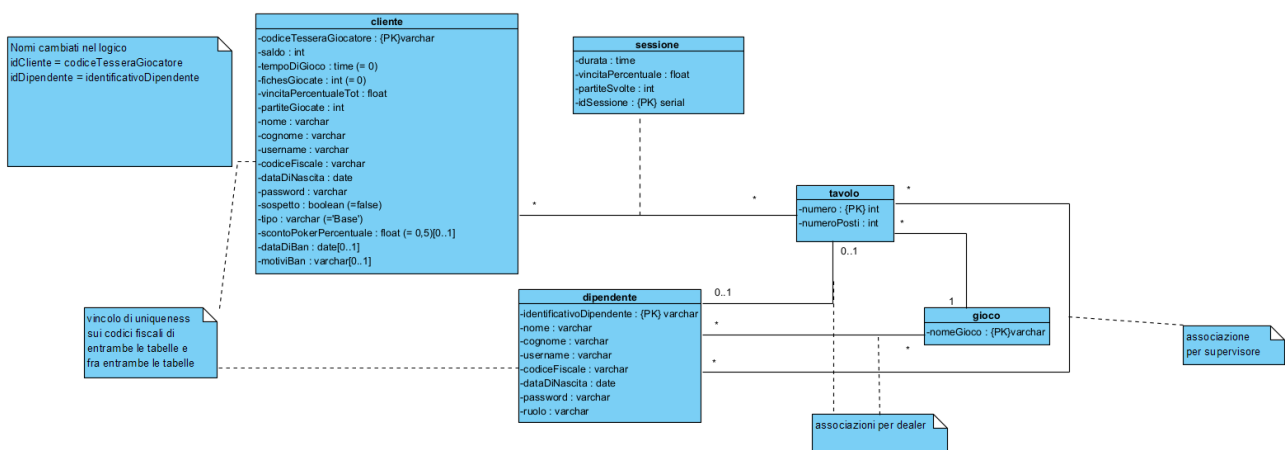
(**nota a lavoro concluso:** alla fine carichiamo i tavoli solo per supervisore, al dealer, se è assegnato ad un tavolo, vengono solo caricate le sessioni relative al suo tavolo e chiaramente gli viene mostrato il tavolo a cui è assegnato)

Se accesso è effettuato da Supervisore:

- OPS1: dopo il login si caricano tutti i clienti sospetti (frequenza media)
(**nota a lavoro concluso:** alla fine carichiamo tutti i clienti)
- OPS2: in base a filtri su cliente, tavolo, data, etc. può visualizzare sessioni (frequenza alta)
- OPS3: ricerca cliente (frequenza medio alta)
(**nota a lavoro concluso:** avendo deciso di caricare tutti i clienti la ricerca cliente si svolge in locale e non interpella il database, OPS2 e OPS3 non esistono più)
- OPS4: ban del cliente (frequenza medio bassa)
- OPS5: aggiunta nuovo account supervisore o dealer (frequenza molto bassa)
- OPS6: assegnare dipendenti a tavoli (frequenza alta), comporta che un trigger verifichi se effettivamente questo sia compatibile col tavolo a cui sta per essere assegnato, cioè il dealer deve saper servire al gioco per cui è predisposto il tavolo
- OPS7: eliminazione account supervisore o dealer (frequenza molto bassa)

Ristrutturato

- accorpamento utente nelle figlie: perché diverse operazioni fanno differenza tra cliente e dipendente (tutte le operazioni OPS del supervisore; OP1, OP2 e OPC1 perché esclusive per cliente), perché utente ha 0 relazioni mentre le sue figlie ne hanno più, perché l'identificativo del cliente è diverso da quello del dipendente (avere le due entità separate rende più semplice controllare la validità dell'identificativo). Lo svantaggio che si ha per questa scelta è per quanto riguarda il login (OP3), per trovare l'utente corretto bisogna scorrere sia cliente che dipendente.
- accorpamento cliente premium in cliente: perché cliente premium non è coinvolto in relazioni, perché non ci sono operazioni sul DB che fanno distinzione tra clienti base e clienti premium.
- accorpamento dealer e supervisore in dipendente: perché dealer e supervisore non hanno molti attributi, ci sono operazioni che non fanno differenza tra supervisore e dealer (OPS5 e OPS7 riguardano genericamente dipendente), l'unico problema è potenzialmente OPS6 in cui i dealer hanno dei vincoli in più rispetto ai supervisori per quanto riguarda l'assegnazione ai tavoli.
- attributo multivalore giochiDoveServe diventa nuova entità: per OPS6 bisogna poter estrarre singolarmente i valori di "giochiDoveServe", attributo di dealer.
- accorpamento di ban in cliente: quando accediamo ad utente vogliamo sempre accedere alle informazioni sul ban, sia che sia il cliente in primo luogo ad accedere (un giocatore bannato non può aprire sessioni di gioco) sia che sia il dealer a voler visualizzare le info dei clienti



Schema Logico

Cliente(codiceTesseraGiocatore, saldo, tempoDiGioco, fichesGiocate, vincitaPercentualeTot, partiteGiocate, nome, cognome, username, codiceFiscale, dataDiNascita, password, tipo, scontoPokerPercentuale, sospetto, dataDiBan, motiviBan)

Sessione(idSessione, durata, vincitaPercentuale, partiteSvolte, *idCliente*, *idTavolo*)

Tavolo(numero, numeroPosti, *idDealer*, *gioco*)

Dipendente(idDipendente, nome, cognome, username, codiceFiscale, dataDiNascita, password, tipologia)

Gioco(nomeGioco)

GiochiDealer(*idGioco*, *idDealer*)

SupervisoreTavolo(*idSupervisore*, *idTavolo*)

FK: in corsivo e rosso

PK: sottolineato

Lo schema logico è stato ottenuto dalla traduzione dello schema concettuale ristrutturato nel modello relazionale:

- Le tabelle Cliente, Dipendente, Tavolo, Sessione e Gioco sono dotate di una chiave primaria che identifica univocamente ogni tupla, mentre le tabelle GiochiDealer e SupervisoreTavolo, essendo utilizzate per rappresentare associazioni tra altre relazioni, sono invece identificate da chiavi composte costituite dalle rispettive chiavi esterne.
- Cliente, Dipendente e Gioco sono tabelle indipendenti, mentre Sessione, GiochiDipendenti e SupervisoreTavolo dipendono dall'esistenza di record in altre tabelle attraverso l'utilizzo di chiavi esterne. In inserimento, modifica e cancellazione (DML) sarà importante da tener a mente l'ordine delle tabelle coinvolte, in questo modo viene garantita l'integrità referenziale del database, impedendo la creazione di riferimenti verso record inesistenti.
- Nello schema è stato previsto che la chiave esterna idDipendente della tabella Tavolo, possa assumere valore NULL, così da rappresentare un tavolo privo di un dealer assegnato, nel caso in cui il tavolo sia dedicato a una slot machine, non è necessario il dealer.

SQL

```
-- Crea tabella Cliente
CREATE TABLE Cliente (
    idCliente VARCHAR(20) PRIMARY KEY,
    --Attributi del giocatore
    saldo INT NOT NULL CHECK(saldo >= 0) DEFAULT 50,
    tempoDiGioco BIGINT NOT NULL DEFAULT 0, --time è limitato nel rappresentare
una durata non oltre le 24 h
    fichesGiocate INT NOT NULL DEFAULT 0 CHECK(fichesGiocate >= 0),
    vincitaPercentualeTot float8 NOT NULL CHECK(vincitaPercentualeTot >= 0 and
vincitaPercentualeTot <= 100) DEFAULT 0,
    partiteGiocate INT NOT NULL CHECK(partiteGiocate >= 0) DEFAULT 0,
    tipo VARCHAR(7) NOT NULL check(tipo in('Base', 'Premium')) DEFAULT 'Base',
--Base/Premium
    scontoPokerPercentuale float8
    CHECK(scontoPokerPercentuale >= 0 and scontoPokerPercentuale <= 1) DEFAULT
0, --non da 1 a 100 così è già pronto per il calcolo
    --Eventuale Ban e indicie sospetto
    sospetto BOOLEAN NOT NULL DEFAULT false,
    dataDiBan DATE DEFAULT null,
    motiviBan VARCHAR(100) DEFAULT null,
    --Attributi del cliente anagrafico
    nome VARCHAR(20) NOT NULL,
    cognome VARCHAR(20) NOT NULL,
    codiceFiscale VARCHAR(16) NOT NULL unique,
    dataDiNascita DATE NOT NULL,
    --Dati d'accesso
    username VARCHAR(20) NOT NULL UNIQUE,
    password VARCHAR(20) NOT NULL,

    check((motiviBan is not null and dataDiBan is not null) or
(motiviBan is null and dataDiBan is null)),
    check(scontoPokerPercentuale = 0 or tipo = 'Premium')
);
```

```

-- Crea Dipendente
CREATE TABLE Dipendente (
    idDipendente VARCHAR(20) PRIMARY KEY,
    --Attributi del dipendente anagrafico
    nome VARCHAR(20) NOT NULL,
    cognome VARCHAR(20) NOT NULL,
    dataDiNascita DATE NOT NULL,
    codiceFiscale VARCHAR(16) NOT NULL unique,
    --Dati d'accesso
    username VARCHAR(20) NOT NULL UNIQUE,
    password VARCHAR(20) NOT NULL,
    --Attributi del dipendente lavorativo
    ruolo VARCHAR(11) NOT NULL check(ruolo in('Supervisore', 'Dealer'))
);

--Crea Gioco
CREATE TABLE Gioco (
    nomeGioco VARCHAR(11) PRIMARY KEY check(nomeGioco in('Poker', 'SlotMachine',
'Blackjack')) --Poker/SlotMachine/Blackjack
);

--Crea GiochiDealer
CREATE TABLE GiochiDealer (
    idDealer VARCHAR(20),
    idGioco VARCHAR(11),
    PRIMARY KEY (idDealer, idGioco),
    FOREIGN KEY (idDealer) REFERENCES Dipendente(IdDipendente) ON DELETE CASCADE
ON UPDATE CASCADE,
    FOREIGN KEY (idGioco) REFERENCES Gioco(nomeGioco) ON DELETE CASCADE
);

--Crea Tavolo
CREATE TABLE Tavolo (
    numero INT PRIMARY KEY check(numero >= 0),
    numeroPosti INT NOT NULL check(numeroPosti > 0),
    idDealer VARCHAR(20) UNIQUE,
    gioco VARCHAR(11) NOT NULL,
    FOREIGN KEY (idDealer) REFERENCES Dipendente(IdDipendente) ON DELETE SET
NULL ON UPDATE CASCADE,
    FOREIGN KEY (gioco) REFERENCES Gioco(nomeGioco),
    CHECK ((gioco = 'SlotMachine' AND idDealer IS NULL AND numeroPosti = 1)
    OR (gioco IN ('Poker', 'Blackjack'))
);

--Crea Supervisore
CREATE TABLE SupervisoreTavolo (
    idSupervisore VARCHAR(20),
    idTavolo INT,
    PRIMARY KEY (idSupervisore, idTavolo),
    FOREIGN KEY (idSupervisore) REFERENCES Dipendente(IdDipendente) ON DELETE
CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (idTavolo) REFERENCES Tavolo(Numero) ON DELETE CASCADE
);

--Crea Sessione
CREATE TABLE Sessione(
    idSessione SERIAL PRIMARY KEY,
    idCliente VARCHAR(20) NOT NULL,
    idTavolo INT,
    durata BIGINT NOT NULL DEFAULT 0,
    vincitaPercentuale float8 NOT NULL check(vincitaPercentuale >= 0 and
vincitaPercentuale <= 100),
    partiteSvolte INT NOT NULL check(partiteSvolte >= 0),

```

```

    FOREIGN KEY (idCliente) REFERENCES Cliente(IdCliente) ON DELETE CASCADE ON
UPDATE CASCADE,
    FOREIGN KEY (idTavolo) REFERENCES Tavolo(Numero) ON DELETE SET NULL
);

```

Il primo passo è definire col DDL, come si fa per il DML, prima le relazioni forti poi quelle deboli, l'ordine è quello presente sopra nel codice. Le parti importanti da tenere a mente sono la scelta dei vincoli (NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY-REFERENCES, CHECK e DEFAULT):

- abbiamo posto il NOT NULL in tutti casi in cui il dato era necessario per l'integrità del DB;
- abbiamo posto l'UNIQUE per garantire che fossero unici i dati inseriti;
- abbiamo posto la PRIMARY KEY in ogni relazione;
- abbiamo posto le FOREIGN KEY-REFERENCES nelle entità dipendenti per collegare le relazioni tra loro;
- abbiamo posto il CHECK per garantire che i dati fossero coerenti (Saldo >=0) e integri (tavolo col gioco SlotMachine non ha dealer);
- abbiamo posto il DEFAULT nei casi in cui gli attributi posseggono un valore iniziale comune (La durata iniziale di una sessione o il saldo iniziale di un giocatore), permettendo di semplificare l'inserimento dei dati e mantenendo comunque la coerenza del database.

Vincoli resi con trigger function

1. vincolo di unicità del codice fiscale e dell'username tra la tabella cliente e quella dipendente
2. solo dealer può essere associato a gioco
3. distinguere associazione tavolo con dealer e tavolo con supervisore

```

1. CREATE or REPLACE FUNCTION noDuplicatiUsernameCliente()
   RETURNS TRIGGER
   LANGUAGE 'plpgsql' as $$
   declare val BOOLEAN;
   BEGIN
       val := EXISTS(
           select *
           from Dipendente as d
           where d.username = new.username
       );

       if(val) THEN
           raise exception 'username già preso';
       end IF;

       return new;
   end;
   $$;

   create TRIGGER usernameCliente
   before insert or UPDATE of username
   on Cliente
   for each row
   execute function noDuplicatiUsernameCliente();

   CREATE or REPLACE FUNCTION noDuplicatiUsernameDipendente()
   RETURNS TRIGGER
   LANGUAGE 'plpgsql' as $$
   declare val BOOLEAN;
   BEGIN
       val := EXISTS(
           select *
           from Cliente as d
           where d.username = new.username
       );

       if(val) THEN
           raise exception 'username già preso';

```

```

        end IF;

        return new;
end;
$$;

create TRIGGER usernameDipendente
before insert or UPDATE of username
on Dipendente
for each row
execute function noDuplicatiUsernameDipendente();

CREATE or REPLACE FUNCTION noDuplicatiCFCliente()
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
declare val BOOLEAN;
BEGIN
    val := EXISTS(
        select *
        from Dipendente as d
        where d.codiceFiscale = new.codiceFiscale
    );

    if(val) THEN
        raise exception 'codice fiscale già preso';
    end IF;

    return new;
end;
$$;

create TRIGGER CFCliente
before insert or UPDATE of codiceFiscale
on Cliente
for each row
execute function noDuplicatiCFCliente();

CREATE or REPLACE FUNCTION noDuplicatiCFDipendente()
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
declare val BOOLEAN;
BEGIN
    val := EXISTS(
        select *
        from Cliente as d
        where d.codiceFiscale = new.codiceFiscale
    );

    if(val) THEN
        raise exception 'codice fiscale già preso';
    end IF;

    return new;
end;
$$;

create TRIGGER CFDipendente
before insert or UPDATE of codiceFiscale
on Dipendente
for each row
execute function noDuplicatiCFDipendente();

```

```

2. CREATE or REPLACE FUNCTION dealerGioco()
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
declare val BOOLEAN;
BEGIN
    val := new.idDealer in(
        select d.idDipendente
        from Dipendente as d
        where d.ruolo = 'Supervisore'
    );

    if(val) THEN
        raise exception 'non è un dealer';
    end IF;

    return new;
end;
$$;

create TRIGGER relazioneDipGioco
before insert or UPDATE
on giochiDealer
for each row
execute function dealerGioco();

```

```

3. CREATE or REPLACE FUNCTION tavoloDealer()
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
declare val BOOLEAN;
BEGIN
    val := new.idDealer in(
        select d.idDipendente
        from Dipendente as d
        where d.ruolo = 'Supervisore'
    );

    if(val) THEN
        raise exception 'non è un dealer';
    end IF;

    return new;
end;
$$;

create TRIGGER relazioneDealerTavolo
before insert or UPDATE of idDealer
on Tavolo
for each row
execute function tavoloDealer();

CREATE or REPLACE FUNCTION tavoloSupervisore()
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
declare val BOOLEAN;
BEGIN
    val := new.idSupervisore in(
        select d.idDipendente
        from Dipendente as d
        where d.ruolo = 'Dealer'
    );

    if(val) THEN
        raise exception 'non è un Supervisore';
    end IF;

    return new;
end;
$$;

```

```

        end IF;

        return new;
    end;
$$;

create TRIGGER relazioneSupervisoreTavolo
before insert or UPDATE of idSupervisore
on SupervisoreTavolo
for each row
execute function tavoloSupervisore();

```

Altre trigger function

In aggiunta a queste è stato utile creare una trigger function in grado di settare a null l'idDealer di un tavolo nel caso in cui questo venisse convertito da tavolo da blackjack a poker o viceversa e il dealer non sapesse servire per quel gioco, in applicativo non è stata sviluppata questa logica.

```

CREATE or REPLACE FUNCTION nonCompatibilitaTavoloDealer() --per
quanto riguarda i giochi
RETURNS TRIGGER
LANGUAGE 'plpgsql' as $$
BEGIN
    if(new.idDealer is not null and new.gioco not in(
        select idGioco
        from giochiDealer
        where idDealer = new.idDealer
    )) THEN
        new.idDealer := null;
    end if;

    RETURN new;
end;
$$;

create TRIGGER tavoloDealer
before UPDATE of gioco
on tavolo
for each row
execute function nonCompatibilitaTavoloDealer();

```